

Design Your Evaluation Pipeline

The success of an AI application often hinges on the ability to differentiate good outcomes from bad outcomes. To be able to do this, you need an evaluation pipeline that you can rely upon. With an explosion of evaluation methods and techniques, it can be confusing to pick the right combination for your evaluation pipeline. This section focuses on evaluating open-ended tasks. Evaluating close-ended tasks is easier, and its pipeline can be inferred from this process.

Step 1. Evaluate All Components in a System

Real-world AI applications are complex. Each application might consist of many components, and a task might be completed after many turns. Evaluation can happen at different levels: per task, per turn, and per intermediate output.

You should evaluate the end-to-end output and each component's intermediate output independently. Consider an application that extracts a person's current employer from their resume PDF, which works in two steps:

1. Extract all the text from the PDF.
2. Extract the current employer from the extracted text.

If the model fails to extract the right current employer, it can be because of either step. If you don't evaluate each component independently, you don't know exactly where your system fails. The first PDF-to-text step can be evaluated using similarity between the extracted text and the ground truth text. The second step can be evaluated using accuracy: given the correctly extracted text, how often does the application correctly extract the current employer?

If applicable, evaluate your application both per turn and per task. A turn can consist of multiple steps and messages. If a system takes multiple steps to generate an output, it's still considered a turn.

Generative AI applications, especially chatbot-like applications, allow back-and-forth between the user and the application, as in a conversation, to accomplish a task. Imagine you want to use an AI model to debug why your Python code is failing. The model responds by asking for more information about your hardware or the Python version you're using. Only after you've provided this information can the model help you debug.

Turn-based evaluation evaluates the quality of each output. *Task-based* evaluation evaluates whether a system completes a task. Did the application help you fix the bug? How many turns did it take to complete the task? It makes a big difference if a system is able to solve a problem in two turns or in twenty turns.

Given that what users really care about is whether a model can help them accomplish their tasks, task-based evaluation is more important. However, a challenge of task-based evaluation is it can be hard to determine the boundaries between tasks. Imagine a conversation you have with ChatGPT. You might ask multiple questions at the same time. When you send a new query, is this a follow-up to an existing task or a new task?

One example of task-based evaluation is the `twenty_questions` benchmark, inspired by the classic game Twenty Questions, in the [BIG-bench benchmark suite](#). One instance of the model (Alice) chooses a concept, such as apple, car, or computer. Another instance of the model (Bob) asks Alice a series of questions to try to identify this concept. Alice can only answer yes or no. The score is based on whether Bob successfully guesses the concept, and how many questions it takes for Bob to guess it. Here's an example of a plausible conversation in this task, taken from the [BIG-bench's GitHub repository](#):

```
Bob: Is the concept an animal?
Alice: No.
Bob: Is the concept a plant?
Alice: Yes.
Bob: Does it grow in the ocean?
Alice: No.
Bob: Does it grow in a tree?
Alice: Yes.
Bob: Is it an apple?
[Bob's guess is correct, and the task is completed.]
```

Step 2. Create an Evaluation Guideline

Creating a clear evaluation guideline is the most important step of the evaluation pipeline. An ambiguous guideline leads to ambiguous scores that can be misleading. If you don't know what bad responses look like, you won't be able to catch them.

When creating the evaluation guideline, it's important to define not only what the application should do, but also what it shouldn't do. For example, if you build a customer support chatbot, should this chatbot answer questions unrelated to your product, such as about an upcoming election? If not, you need to define what inputs are out of the scope of your application, how to detect them, and how your application should respond to them.

Define evaluation criteria

Often, the hardest part of evaluation isn't determining whether an output is good, but rather what good means. In retrospect of one year of deploying generative AI applications, [LinkedIn](#) shared that the first hurdle was in creating an evaluation guideline. *A correct response is not always a good response.* For example, for their AI-powered Job Assessment application, the response “You are a terrible fit” might be correct but not helpful, thus making it a bad response. A good response should explain the gap between this job's requirements and the candidate's background, and what the candidate can do to close this gap.

Before building your application, think about what makes a good response. [LangChain's State of AI 2023](#) found that, on average, their users used 2.3 different types of feedback (criteria) to evaluate an application. For example, for a customer support application, a good response might be defined using three criteria:

1. Relevance: the response is relevant to the user's query.
2. Factual consistency: the response is factually consistent with the context.
3. Safety: the response isn't toxic.

To come up with these criteria, you might need to play around with test queries, ideally real user queries. For each of these test queries, generate multiple responses, either manually or using AI models, and determine if they are good or bad.

Create scoring rubrics with examples

For each criterion, choose a scoring system: would it be binary (0 and 1), from 1 to 5, between 0 and 1, or something else? For example, to evaluate whether an answer is consistent with a given context, some teams use a binary scoring system: 0 for factual inconsistency and 1 for factual consistency. Some teams use three values: -1 for contradiction, 1 for entailment, and 0 for neutral. Which scoring system to use depends on your data and your needs.

On this scoring system, create a rubric with examples. What does a response with a score of 1 look like and why does it deserve a 1? Validate your rubric with humans: yourself, coworkers, friends, etc. If humans find it hard to follow the rubric, you need to refine it to make it unambiguous. This process can require a lot of back and forth, but it's necessary. A clear guideline is the backbone of a reliable evaluation pipeline. This guideline can also be reused later for training data annotation, as discussed in [Chapter 8](#).

Tie evaluation metrics to business metrics

Within a business, an application must serve a business goal. The application's metrics must be considered in the context of the business problem it's built to solve.

For example, if your customer support chatbot's factual consistency is 80%, what does it mean for the business? For example, this level of factual consistency might make the chatbot unusable for questions about billing but good enough for queries about product recommendations or general customer feedback. Ideally, you want to map evaluation metrics to business metrics, to something that looks like this:

- Factual consistency of 80%: we can automate 30% of customer support requests.
- Factual consistency of 90%: we can automate 50%.
- Factual consistency of 98%: we can automate 90%.

Understanding the impact of evaluation metrics on business metrics is helpful for planning. If you know how much gain you can get from improving a certain metric, you might have more confidence to invest resources into improving that metric.

It's also helpful to determine the usefulness threshold: what scores must an application achieve for it to be useful? For example, you might determine that your chatbot's factual consistency score must be at least 50% for it to be useful. Anything below this makes it unusable even for general customer requests.

Before developing AI evaluation metrics, it's crucial to first understand the business metrics you're targeting. Many applications focus on *stickiness* metrics, such as daily, weekly, or monthly active users (DAU, WAU, MAU). Others prioritize *engagement* metrics, like the number of conversations a user initiates per month or the duration of each visit—the longer a user stays on the app, the less likely they are to leave. Choosing which metrics to prioritize can feel like balancing profits with social responsibility. While an emphasis on stickiness and engagement metrics can lead to higher revenues, it may also cause a product to prioritize addictive features or extreme content, which can be detrimental to users.

Step 3. Define Evaluation Methods and Data

Now that you’ve developed your criteria and scoring rubrics, let’s define what methods and data you want to use to evaluate your application.

Select evaluation methods

Different criteria might require different evaluation methods. For example, you use a small, specialized toxicity classifier for toxicity detection, semantic similarity to measure relevance between the response and the user’s original question, and an AI judge to measure the factual consistency between the response and the whole context. An unambiguous scoring rubric and examples will be critical for specialized scorers and AI judges to succeed.

It’s possible to mix and match evaluation methods for the same criteria. For example, you might have a cheap classifier that gives low-quality signals on 100% of your data, and an expensive AI judge to give high-quality signals on 1% of the data. This gives you a certain level of confidence in your application while keeping costs manageable.

When logprobs are available, use them. Logprobs can be used to measure how confident a model is about a generated token. This is especially useful for classification. For example, if you ask a model to output one of the three classes and the model’s logprobs for these three classes are all between 30 and 40%, this means the model isn’t confident about this prediction. However, if the model’s probability for one class is 95%, this means that the model is highly confident about this prediction. Logprobs can also be used to evaluate a model’s perplexity for a generated text, which can be used for measurements such as fluency and factual consistency.

Use automatic metrics as much as possible, but don’t be afraid to fall back on human evaluation, even in production. Having human experts manually evaluate a model’s quality is a long-standing practice in AI. Given the challenges of evaluating open-ended responses, many teams are looking at human evaluation as the North Star metric to guide their application development. Each day, you can use human experts to evaluate a subset of your application’s outputs that day to detect any changes in the application’s performance or unusual patterns in usage. For example, [LinkedIn](#) developed a process to manually evaluate up to 500 daily conversations with their AI systems.

Consider evaluation methods to be used not just during experimentation but also during production. During experimentation, you might have reference data to compare your application’s outputs to, whereas, in production, reference data might not be immediately available. However, in production, you have actual users. Think about what kinds of feedback you want from users, how user feedback correlates to other evaluation metrics, and how to use user feedback to improve your application. How to collect user feedback is discussed in [Chapter 10](#).

Annotate evaluation data

Curate a set of annotated examples to evaluate your application. You need annotated data to evaluate each of your system’s components and each criterion, for both turn-based and task-based evaluation. Use actual production data if possible. If your application has natural labels that you can use, that’s great. If not, you can use either humans or AI to label your data. [Chapter 8](#) discusses AI-generated data. The success of this phase also depends on the clarity of the scoring rubric. The annotation guideline created for evaluation can be reused to create instruction data for finetuning later, if you choose to finetune.

Slice your data to gain a finer-grained understanding of your system. Slicing means separating your data into subsets and looking at your system’s performance on each subset separately. I wrote at length about slice-based evaluation in *Designing Machine Learning Systems* (O’Reilly), so here, I’ll just go over the key points. A finer-grained understanding of your system can serve many purposes:

- Avoid potential biases, such as biases against minority user groups.
- Debug: if your application performs particularly poorly on a subset of data, could that be because of some attributes of this subset, such as its length, topic, or format?
- Find areas for application improvement: if your application is bad on long inputs, perhaps you can try a different processing technique or use new models that perform better on long inputs.
- Avoid falling for [Simpson’s paradox](#), a phenomenon in which model A performs better than model B on aggregated data but worse than model B on every subset of data. [Table 4-6](#) shows a scenario where model A outperforms model B on each subgroup but underperforms model B overall.

Table 4-6. An example of Simpson’s paradox.^a

	Group 1	Group 2	Overall
Model A	93% (81/87)	73% (192/263)	78% (273/350)
Model B	87% (234/270)	69% (55/80)	83% (289/350)

^a I also used this example in *Designing Machine Learning Systems*. Numbers from Charig et al., “Comparison of Treatment of Renal Calculi by Open Surgery, Percutaneous Nephrolithotomy, and Extracorporeal Shockwave Lithotripsy”, *British Medical Journal (Clinical Research Edition)* 292, no. 6524 (March 1986): 879–82.

You should have multiple evaluation sets to represent different data slices. You should have one set that represents the distribution of the actual production data to estimate how the system does overall. You can slice your data based on tiers (paying users versus free users), traffic sources (mobile versus web), usage, and more. You can have a set consisting of the examples for which the system is known to frequently

make mistakes. You can have a set of examples where users frequently make mistakes—if typos are common in production, you should have evaluation examples that contain typos. You might want an out-of-scope evaluation set, inputs your application isn't supposed to engage with, to make sure that your application handles them appropriately.

If you care about something, put a test set on it. The data curated and annotated for evaluation can then later be used to synthesize more data for training, as discussed in [Chapter 8](#).

How much data you need for each evaluation set depends on the application and evaluation methods you use. In general, the number of examples in an evaluation set should be large enough for the evaluation result to be reliable, but small enough to not be prohibitively expensive to run.

Let's say you have an evaluation set of 100 examples. To know whether 100 is sufficient for the result to be reliable, you can create multiple bootstraps of these 100 examples and see if they give similar evaluation results. Basically, you want to know that if you evaluate the model on a different evaluation set of 100 examples, would you get a different result? If you get 90% on one bootstrap but 70% on another bootstrap, your evaluation pipeline isn't that trustworthy.

Concretely, here's how each bootstrap works:

1. Draw 100 samples, with replacement, from the original 100 evaluation examples.
2. Evaluate your model on these 100 bootstrapped samples and obtain the evaluation results.

Repeat for a number of times. If the evaluation results vary wildly for different bootstraps, this means that you'll need a bigger evaluation set.

Evaluation results are used not just to evaluate a system in isolation but also to compare systems. They should help you decide which model, prompt, or other component is better. Say a new prompt achieves a 10% higher score than the old prompt—how big does the evaluation set have to be for us to be certain that the new prompt is indeed better? In theory, a statistical significance test can be used to compute the sample size needed for a certain level of confidence (e.g., 95% confidence) if you know the score distribution. However, in reality, it's hard to know the true score distribution.



OpenAI suggested a rough estimation of the number of evaluation samples needed to be certain that one system is better, given a score difference, as shown in **Table 4-7**. A useful rule is that for every 3× decrease in score difference, the number of samples needed increases 10×.²⁸

Table 4-7. A rough estimation of the number of evaluation samples needed to be 95% confident that one system is better. Values from OpenAI.

Difference to detect	Sample size needed for 95% confidence
30%	~10
10%	~100
3%	~1,000
1%	~10,000

As a reference, among evaluation benchmarks in **Eleuther’s lm-evaluation-harness**, the median number of examples is 1,000, and the average is 2,159. The organizers of the **Inverse Scaling prize** suggested that 300 examples is the absolute minimum and they would prefer at least 1,000, especially if the examples are being synthesized (**McKenzie et al., 2023**).

Evaluate your evaluation pipeline

Evaluating your evaluation pipeline can help with both improving your pipeline’s reliability and finding ways to make your evaluation pipeline more efficient. Reliability is especially important with subjective evaluation methods such as AI as a judge.

Here are some questions you should be asking about the quality of your evaluation pipeline:

Is your evaluation pipeline getting you the right signals?

Do better responses indeed get higher scores? Do better evaluation metrics lead to better business outcomes?

How reliable is your evaluation pipeline?

If you run the same pipeline twice, do you get different results? If you run the pipeline multiple times with different evaluation datasets, what would be the variance in the evaluation results? You should aim to increase reproducibility and reduce variance in your evaluation pipeline. Be consistent with the configurations of your evaluation. For example, if you use an AI judge, make sure to set your judge’s temperature to 0.

²⁸ This is because the square root of 10 is approximately 3.3.